



Object Oriented Programming: Python BootCamp

HIT-Emily on BSc Program
April-May, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 06 : File I/O

So far, you've written programs that get their input from one of two places: from the program itself or from the user. Program output has been limited to the interactive window.

In this chapter, we will see how to use actual data files.

Recap (The File System Hierarchy):

File systems organize files in a hierarchy of **directories**, which are also known as **folders**. At the top of the hierarchy is a directory called the **root** directory.

Each file in directory has a **file name** that must be unique from any other file in the same directory. Directories can also contain other directories, called **subdirectories** or **subfolders**.

To locate a file in a file system, you can list the directories in order, starting with the root directory, followed by the name of the file. A string with the file location represented in this manner is called a **file path**.

How you write file paths depends on your operating system. Here are three examples of file paths on Windows, macOS, and Linux:

1. **Windows:** C:\Users\David\Documents\hello.txt
2. **macOS:** /Users/David/Documents/hello.txt
3. **Ubuntu Linux:** /home/David/Documents/hello.txt

Text Files

The open() Function

Before you can read or write a file, you have to open it using Python's built-in open() function. This function creates a **file object**.

You have learned about **objects** in the previous lecture.

```
In [ ]: my_file = open(file='data/sample.txt', mode='w')
```

mode: This parameter is a string that is used to specify the mode in which the file is to be opened.

- "r": This string is used to read(only) the file.
- "w": This string is used for writing on/over the file. **If the file with the supplied name doesn't exist, it creates one for you.**
- "a": This string is used to add(append) content to an existing file. **If no such file exists, it creates one for you.**

Let's check some file attributes:

```
In [ ]: print("Name of the file: ", my_file.name)
print("Closed or not : ", my_file.closed)
print("Opening mode : ", my_file.mode)
```

The close() Function

The close() method of a file object flushes any unwritten information and closes the file object, after which no more writing can be done.

```
In [ ]: my_file.close()
```

The write() Function

The write() method writes any string to an open file.

```
In [ ]: my_file = open(file='data/sample.txt', mode='w')
my_file.write('Hello file!')
my_file.write('\n')
my_file.write('This is me!')
my_file.close()
```

The write() method does not add a newline character ('\n') to the end of the string.

The read() Method

The read() method reads a string from an open file.

```
In [ ]: my_file = open(file='data/sample.txt', mode='r')
file_output = my_file.read()
my_file.close()
```

```
In [ ]: file_output
```

```
In [ ]: my_file = open(file='data/sample.txt', mode='r')
file_output = my_file.readline()
my_file.close()
```

```
In [ ]: file_output
```

```
In [ ]: my_file = open(file='data/sample.txt', mode='r')
file_output = my_file.readlines()
my_file.close()
```

```
In [ ]: file_output
```

```
In [ ]: my_file = open(file='data/sample.txt', mode='r')
file_output = ""

for line in my_file:
    print(line)

my_file.close()
```

```
In [ ]: file_output
```

EXERCISES 05 - PART ONE

CSV Files

Now we will talk about .csv files. These are really wide spread and useful type of files. **CSV** is short for **Comma Separated Values** and as the name suggests, this is a really simple but powerful way of storing your data.

```
In [ ]: import csv

# open a csv file and using the csv reader function
# also setting skipinitialspace true, so that commas in our "values" are ignored.

csv_file = open("data/sample.csv", "r")
reader = csv.reader(csv_file, skipinitialspace=True)

for row in reader:
    print(row)

csv_file.close()
```

```
In [ ]: # same but open csv data in dictionary format
csv_file = open("data/sample.csv", "r")
reader = csv.DictReader(csv_file)

for row in reader:
    print(row)

csv_file.close()
```

```
In [ ]: # write csv ourselves
# overwrites current csv file, use "a" for append instead to simply write into existing file.

csv_file = open("data/sample.csv", "w")
writer = csv.writer(csv_file)

writer.writerow(['5', 'Rose', '30', 'Grenoble'])
writer.writerow(['6', 'Nona', '23', 'Toulouse'])

csv_file.close()
```

SELF-STUDY

Towards Better Programming and Debugging

Error Types:

This section is about different error types. So an error is not simply an error, but can be differentiated into **errors** and **exceptions**.

When an error occurs the script will not run, for example when forgetting to close a bracket or a colon in your for loop will raise a syntax error.

These errors are "caught" by your IDE even before executing your script, since these are stopping your interpreter from running the script properly.

And then there are exceptions which "turn" into errors when we don't handle them properly, for example wanting to access the 3rd element of a list that only has 2 elements will raise the "IndexError" and tell you that the list index is out of range.

Some common errors are:

```
In [1]: # variable name error
3_apples = ['apple', 'apple', 'apple']

Cell In[1], line 2
      3_apples = ['apple', 'apple', 'apple']
      ^
SyntaxError: invalid decimal literal
```

```
In [2]: # forgot to close the brackets
three_apples = ['apple', 'apple', 'apple'

Cell In[2], line 2
      three_apples = ['apple', 'apple', 'apple'
      ^
SyntaxError: incomplete input
```

```
In [ ]: # forgot to place colon and wrong indentation
for i in range(3)
    print(three_apples[i])
    print("hello")
```

```
In [ ]: # wrong index
three_apples = ['A','B','C']
for i in range(4):
    print(three_apples[i])
```

```
In [ ]: # difference in local and global variables
# or wanting to use functions or modules that haven't been created or imported yet
```

```
In [ ]: # problems with data types
converting_data_types = "I am a string of characters"
print(float(converting_data_types))
print("hello " + 6)
```

Debugging and Exceptions:

This section is about debugging techniques and using the try and except methods when there is the chance of an error being raised.

First use the print function to keep track of your variables and outputs of functions, so you eventually catch the point where your code misbehaves.

Secondly we will introduce the try and except functions that act (kind of) like the if and else functions you are already familiar with but with exceptions being raised. So when the try function fails the except function will be called.

```
In [ ]: user_input = input("tell me your name please: ")

try:
    user_input = float(user_input)
    print("that is a number, not your name!")

except:
    print("thank you", user_input)
```

when an exception is raised in the try block, then the except block will be called. e.g. the user_input is a string, then casting float would result in an error and thus the except block.

Better Programming:

As we've already mentioned a few times, writing nice, easy to understand code should not be underestimated in its importance. There are conventions in place that you can refer to to make sure your code is good. For python, the official style guide you should refer to when there's any questions is **PEP-8**.

Conventions are usually specified in a very strict, precise manner. For most of them, it makes sense to just follow them as they are defined. But keep in mind that you should not spend more time with the style guide than with your actual code. For example, PEP-8 defines the maximum line length at 79 characters. We will not subtract points if your lines are a bit longer than that, but don't make them overly long either.

Comments and Docstrings

The importance of good, explanatory comments and docstrings can not be overstated. Here's what you should and shouldn't do:

DOs:

- Add explanatory comments
 - Before a line (or group of lines) that are not entirely obvious
 - with the same indentation as the code they describe
 - only rarely in the same line as the code it describes
- Add docstrings
 - as the first line of every function
 - as the first line of your module (i.e. your program)

DO NOTs:

- add comments that are self-explanatory
 - e.g.: x = x + 1 # increment x
- add comments that are wrong
 - this is, of course, worse than leaving no comment at all

These rules should be detailed enough for this course. You can find more detailed descriptions, as well as some examples for what your code should/shouldn't look like, in the section on comments of **PEP-8**.

Whitespaces

There's a lot of very exact rules for how to distribute whitespace throughout your code and comments to increase readability. Here's another list of bullet points:

- DO put spaces:
 - after commas, colons and semicolons
 - around "=", "+=", etc. in assignments
 - around most binary operators, e.g. "2 + 2"
- DO NOT put spaces:
 - between a function name and "("
 - around "=" in keyword arguments
 - after "(" and before ")"

```
Bad:
def my_function ( a,b,c ) :
    some_dict={"a" : "b", 4:5}
    another_function(a+b+c, n = 42)
```

```
Good:
def my_function(a, b, c):
    some_dict = {"a": "b", 4: 5}
    another_function(a + b + c, n=42)
```

Blank Lines

Blank lines and linebreaks in the right places can greatly increase readability - here's how to use them:

- separate function definitions by two blank lines
- separate code blocks (e.g. loops, if-statements) by a blank line
- separate logical segments in your code by line breaks
 - as a rule of thumb: no more than 4 lines without a blank
- break up long lines of code into multiple ones
 - either logically, by using intermediate results saved in variables
 - or syntactically, by breaking at supported points in the line

explicit continuation

```
total_income = gross_income \

    - taxes \
    - student_loan_interest

</code>
```

Naming conventions

Of course, you should follow naming conventions so that it is clear what a name refers to. You need to avoid names that python already uses - like list, str, int, and so on.

- variable and function names should be lower_case_with_underscores
- constants should be UPPER_CASE_WITH_UNDERSCORES
- classes should be UpperCamelCase
- finde a balance between descriptiveness and length
 - good: translation_dict, word_counter
 - bad: a, list_for_storing_all_the_words_of_the_text

```
In [ ]:
```